PROJECT REPORT

Data Structures

MAZE RUNNER

Maze Generation and Solving using DFS and BFS

Implementation of Stack, Queue, 2D Grid and Parent-Pointer Structures in C++

Submitted by

Name	UID
Abhay Singh	25BCS11923
Navraj Singh	25BCS10288

November 2026

1. Abstract

Maze Runner is a C++ application that generates random mazes and solves them with two classic graph-traversal algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS). The project is built to show how core data structures decide the behaviour of an algorithm. A two-dimensional vector stores the maze, a stack drives both maze generation and the DFS solver, a queue drives the BFS solver, and a table of parent pairs lets either solver rebuild the route it found.

The same engine is used by a console game and by a small custom HTTP server that feeds a browser front end. This report concentrates on the data structures: why each one was chosen, how it is declared and used in the code, and what it costs in time and memory. In a test over 1,000 random 21 × 21 mazes, BFS returned a path of 45.1 steps on average, against 70.5 for DFS. DFS was never shorter than BFS, and every maze was solvable.

2. Introduction

A maze is a natural example of a graph. Every open cell is a vertex, and every pair of neighbouring open cells is an edge. Finding a way out is then a graph-search problem, and the data structure that holds the frontier of unexplored cells decides how the search behaves. A queue (first in, first out) spreads the search outward in rings and yields BFS. A stack (last in, first out) pushes the search deep along one corridor before backing up and yields DFS.

Maze Runner puts both side by side on the same maze so that the difference can be seen and measured. The project also reuses DFS in a second role: carving the maze itself.

2.1 Objectives

- Represent a maze with a suitable data structure and treat it as an implicit graph.
- Generate random mazes with the DFS backtracking technique, using recursion (the call stack).
- Solve a maze with BFS using `std::queue` and with DFS using `std::stack`.
- Reconstruct the final route from a parent table and return it as a `vector` of coordinates.
- Compare the two algorithms on path length, memory and correctness.
- Expose the engine through a console interface and a REST API with a web front end.

2.2 Tools and Technologies

Component	Technology
Language / standard	C++17 (g++ / MinGW)
Containers used	std::vector, std::stack, std::queue, std::pair
Randomness	std::mt19937 seeded from std::random_device, std::shuffle
Networking	Winsock2 (ws2_32), a hand-written HTTP/1.1 server on port 3000
Front end	HTML5 Canvas, CSS, JavaScript (fetch API)

Component	Technology
Build	CMake 3.10 or a direct g++ command

3. System Architecture

The code is split into three layers. The engine knows nothing about input or output, so the console program and the server can both reuse it unchanged.

Layer	Files	Responsibility
Engine	src/engine/Maze.h, Maze.cpp	Grid storage, maze generation, BFS, DFS, path reconstruction, player movement
Console client	src/console/main_console.cpp	Menu-driven terminal game: display, play with WASD, solve, regenerate
Server	src/server/HttpServer.h/.cpp, main_server.cpp	Socket loop, routing, JSON serialisation of grid and paths, static file serving
Web client	web/index.html, style.css, script.js	Canvas drawing, keyboard movement, animation of the solution path

Data flow in the web version: the browser calls /api/maze or /api/new; the server asks the Maze object for its grid and turns it into JSON; the browser draws it. When a solver button is pressed, the browser calls /api/solve?algorithm=bfs (or dfs), the server runs the chosen method on the same Maze object and returns the path as a list of [row, col] pairs, which the browser animates.

3.1 The Maze class

```
class Maze {
private:
    int rows, cols;
    std::vector<std::vector<int>> grid;        // 1 = wall, 0 = open
    std::pair<int,int> start, finish, player;  // (row, col) coordinates
    std::mt19937 rng;                          // random engine
    void generateMazeDFS(int r, int c);        // recursive carving
    void addLoops(double removalRatio = 0.12); // braiding
    std::vector<std::pair<int,int>> reconstructPath(
        std::vector<std::vector<std::pair<int,int>>>& parent);
public:
    void generateMaze();
    std::vector<std::pair<int,int>> solveBFS();
    std::vector<std::pair<int,int>> solveDFS();
    ...  // movePlayer, isCompleted, getGrid, getStart, getFinish
};
```

4. Data Structures Used: Overview

The table below lists every data structure in the engine, where it appears and the reason it was chosen.

Data structure	Declaration	Used in	Why this structure
2D array (vector of vectors)	vector<vector<int>> grid	Maze storage	O(1) access to any cell by (row, col); cache-friendly; neighbours are found by adding offsets
Pair	pair<int,int>	Coordinates, start, finish, player	Bundles row and column into one value; supports == and copying
Queue (FIFO)	queue<pair<int,int>> q	solveBFS	Processes cells in order of discovery, which gives level-by-level exploration
Stack (LIFO)	stack<pair<int,int>> st	solveDFS	Always explores the most recently found cell, which gives deep exploration
Call stack (recursion)	generateMazeDFS(r, c)	Maze generation	Backtracking happens automatically when a call returns
Boolean matrix	vector<vector<bool>> visited	BFS, DFS	Marks cells already seen in O(1), which prevents infinite loops
Parent matrix	vector<vector<pair<int,int>>> parent	BFS, DFS	Each cell stores the cell it was reached from, so a path can be traced backwards
Dynamic array	vector<pair<int,int>> path	Path result, wall candidates	Growable list that can be reversed and shuffled
Direction table	vector<pair<int,int>> directions	Generation, BFS, DFS	Replaces four copy-pasted branches with a loop over offsets

5. Maze Representation: 2D Grid as an Implicit Graph

5.1 Grid storage

The maze is a 21 × 21 matrix of integers created with grid.resize(rows, vector<int>(cols, 1)). A value of 1 is a wall and 0 is an open cell. The start is fixed at (1, 1) and the finish at (rows − 2, cols − 2) = (19, 19). Odd rows and columns hold the "rooms" of the maze, even rows and columns hold the walls between them, and the outer border stays solid.

The 21 × 21 size gives 441 cells. Over 1,000 generated mazes, an average of 213.6 cells were open, so roughly half of the grid is corridor.

5.2 Why not an adjacency list?

A general graph would normally be stored as an adjacency list or matrix. Here the graph is implicit: the neighbours of a cell are always the four cells at offsets (−1,0), (1,0), (0,−1) and (0,1), and a neighbour is an edge only if that cell is open. Storing the edges separately would waste memory and add code. The grid is therefore both the data store and the graph.

```
bool Maze::isValid(int r, int c) {
    return r >= 0 && r < rows &&
           c >= 0 && c < cols &&
           grid[r][c] == 0;          // inside bounds and not a wall
}
```

`isValid` is the single "is this edge usable?" test, called by the player movement code, BFS and DFS. Because the bounds check comes before the array access, no call can read outside the matrix.

5.3 Direction table

Each of the four moves is stored as a (row offset, column offset) pair in a `vector<pair<int,int>>`. Looping over this table visits all neighbours with one block of code. The generator uses offsets of ±2 so that it jumps from room to room, while the solvers use offsets of ±1 to walk cell by cell.

```
vector<pair<int,int>> directions = { {-1,0}, {1,0}, {0,-1}, {0,1} };
for (auto d : directions) {
    int nr = current.first  + d.first;
    int nc = current.second + d.second;
    if (isValid(nr, nc) && !visited[nr][nc]) { ... }
}
```

6. Maze Generation: Recursive DFS and Loop Braiding

6.1 Recursive backtracker

Generation starts with a grid full of walls and carves passages with a depth-first walk. From the current room the algorithm picks the four neighbouring rooms in random order. For each one that is still solid, it knocks down the wall in between and recurses into that room. When no unvisited neighbour is left, the function returns, which is the backtracking step.

```
void Maze::generateMazeDFS(int r, int c) {
    grid[r][c] = 0;                          // carve this room
    vector<pair<int,int>> directions = {{-2,0},{2,0},{0,-2},{0,2}};
    shuffle(directions.begin(), directions.end(), rng);
    for (auto d : directions) {
        int nr = r + d.first, nc = c + d.second;
        if (nr > 0 && nr < rows-1 && nc > 0 && nc < cols-1 && grid[nr][nc] == 1) {
```

```
            grid[r + d.first/2][c + d.second/2] = 0;  // remove wall between
            generateMazeDFS(nr, nc);                  // go deeper
        }
    }
}
```

Data structure role. The stack here is the implicit call stack. Each frame holds r, c and the shuffled directions vector, so returning from a call restores exactly the state needed to try the next direction. The deepest possible recursion equals the number of rooms, which is 10 × 10 = 100 frames for a 21 × 21 grid, so stack overflow is not a concern at this size.

Why the result is a tree. Every room is carved exactly once and joined to the room it was reached from, so the carved cells form a spanning tree: no cycles and exactly one route between any two rooms. A perfect maze like this has only one solution, which would make BFS and DFS return the same path.

6.2 Loop braiding

To give the solvers something to compare, addLoops(0.12) adds extra connections. It scans every interior wall cell and keeps it as a candidate if the cells on opposite sides of it (above and below, or left and right) are both open. Removing such a wall joins two corridors and creates a cycle.

```
vector<pair<int,int>> wallCandidates;
for (int r = 1; r < rows-1; r++)
 for (int c = 1; c < cols-1; c++)
   if (grid[r][c] == 1) {
     bool vertical   = (grid[r-1][c] == 0 && grid[r+1][c] == 0);
     bool horizontal = (grid[r][c-1] == 0 && grid[r][c+1] == 0);
     if (vertical || horizontal) wallCandidates.push_back({r, c});
   }
shuffle(wallCandidates.begin(), wallCandidates.end(), rng);
int wallsToRemove = wallCandidates.size() * removalRatio;   // 12%
if (wallsToRemove < 3 && !wallCandidates.empty())
   wallsToRemove = min((int)wallCandidates.size(), 3);     // at least 3
for (int i = 0; i < wallsToRemove; i++)
   grid[wallCandidates[i].first][wallCandidates[i].second] = 0;
```

A vector of candidate coordinates is shuffled and the first 12% (at least three) are opened. Shuffling first makes the choice uniformly random. After braiding the maze is a general graph with cycles, so several routes to the exit exist and the shortest one is no longer obvious. This is what makes BFS and DFS give different answers.

Complexity. Carving visits each room once: O(R·C). Candidate scanning is O(R·C) and the shuffle is O(k) for k candidates.

6.3 Random number generator

std::mt19937 rng is a member of the class and is seeded once in the constructor with random_device. Keeping it as a member means every call to generateMaze() continues the same random sequence, so each new maze differs from the last.

7. BFS Implementation: Queue

Breadth-first search explores the maze in rings of equal distance from the start. It first visits all cells one step away, then all cells two steps away, and so on. Because of this, the first time the finish cell is reached, it has been reached by a route with the fewest possible steps.

7.1 Structures involved

Structure	Type	Role
Frontier	queue<pair<int,int>>	Holds discovered cells waiting to be expanded, oldest first
Seen set	vector<vector<bool>> (rows × cols)	Prevents a cell from being queued twice
Parent table	vector<vector<pair<int,int>>> initialised to {-1,-1}	parent[r][c] = the cell from which (r,c) was first reached; {-1,-1} means not reached

7.2 Code

```
vector<pair<int,int>> Maze::solveBFS() {
  vector<vector<bool>> visited(rows, vector<bool>(cols, false));
  vector<vector<pair<int,int>>> parent(rows, vector<pair<int,int>>(cols, {-1,-1}));
  queue<pair<int,int>> q;

  q.push(start);
  visited[start.first][start.second] = true;

  while (!q.empty()) {
    pair<int,int> current = q.front();  q.pop();
    if (current == finish) return reconstructPath(parent);

    for (auto d : directions) {
      int nr = current.first + d.first, nc = current.second + d.second;
      if (isValid(nr, nc) && !visited[nr][nc]) {
        visited[nr][nc] = true;       // mark when queued, not when popped
        parent[nr][nc] = current;
        q.push({nr, nc});
      }
```

```
        }
    }
    return {};                          // queue empty: no route
}
```

7.3 Why it finds the shortest path

- Cells leave the queue in non-decreasing order of distance from the start, because a cell at distance d is always queued before any cell at distance d + 1.
- A cell is marked visited the moment it is queued, so its parent is fixed by the first (and therefore shortest) route that touches it.
- Following parent pointers from the finish back to the start therefore gives a route of minimum length. All steps have equal cost, so no weights are needed.

Dry-run idea. Imagine a junction two cells before the exit that splits into a long loop and a short corridor. BFS expands both branches one layer at a time, so the short corridor reaches the exit first and the long loop is abandoned without finishing.

Complexity. Each open cell enters the queue once and each of its four neighbours is examined once: O(V + E) = O(R·C) time. The queue can hold a full frontier, plus two R × C tables, so space is O(R·C).

8. DFS Implementation: Stack

Depth-first search follows one corridor as far as it can. When it reaches a dead end, it returns to the most recent junction and tries another branch. A stack gives this behaviour naturally: the newest discovered cell is always expanded next.

```
vector<pair<int,int>> Maze::solveDFS() {
    // visited[] and parent[] declared exactly as in solveBFS
    stack<pair<int,int>> st;
    st.push(start);
    visited[start.first][start.second] = true;

    while (!st.empty()) {
        pair<int,int> current = st.top();  st.pop();
        if (current == finish) return reconstructPath(parent);

        for (auto d : directions) {
            int nr = current.first + d.first, nc = current.second + d.second;
            if (isValid(nr, nc) && !visited[nr][nc]) {
                visited[nr][nc] = true;
                parent[nr][nc] = current;
                st.push({nr, nc});
```

```
        }
      }
    }
    return {};
}
```

The code is the same as BFS with one change: std::stack replaces std::queue. The change in data structure alone changes the order in which cells are expanded, and with it the route found. This is the central point of the project.

8.1 Iterative instead of recursive

The solver uses an explicit std::stack and not recursion. An explicit stack lives on the heap, so a large maze cannot overflow the call stack, and the loop can stop immediately when the finish is popped. (The generator, in contrast, uses recursion because backtracking maps cleanly onto function returns.)

8.2 Behaviour and limits

- The path returned is always a valid route, because every parent pointer was set by a legal step.
- It is not guaranteed to be shortest. Cells are marked visited when pushed, so the first branch that reaches a cell claims it, even if a shorter branch would have arrived later.
- This variant explores neighbours so that the last direction pushed (right, in this code) is expanded first. The order of the directions table therefore biases the route.

Complexity. Same as BFS: O(R·C) time and O(R·C) space for the stack and the two tables.

9. Path Reconstruction

Neither solver stores a path while it searches. Each only fills the parent table. When the finish cell is popped, reconstructPath walks backwards through that table.

```
vector<pair<int,int>> Maze::reconstructPath(
        vector<vector<pair<int,int>>>& parent) {
    vector<pair<int,int>> path;
    pair<int,int> current = finish;
    while (current != start) {
        if (current.first == -1) return {};   // broken chain: unreachable
        path.push_back(current);
        current = parent[current.first][current.second];
    }
    path.push_back(start);
    reverse(path.begin(), path.end());        // finish→start becomes start→finish
    return path;
```

```
}
```

The parent table works as a set of linked lists that all end at the start: every reached cell points to its predecessor. Following the links from the finish costs O(L) for a path of L steps, and reverse costs another O(L). Storing one parent per cell, instead of a full path per queue entry, keeps memory at O(R·C) and not O(R·C·L). The sentinel value {-1,-1} marks a cell that was never reached, so an unreachable finish returns an empty vector, which callers treat as "no path".

The returned vector<pair<int,int>> has size L + 1 (it includes both ends), so the reported path length is path.size() - 1.

10. BFS vs DFS: Comparison and Results

10.1 Theoretical comparison

Property	BFS	DFS
Frontier structure	Queue (FIFO)	Stack (LIFO)
Exploration pattern	Layer by layer, outward from the start	Down one corridor, then back up
Shortest path (unweighted)	Guaranteed	Not guaranteed
Time complexity	O(V + E) = O(R·C)	O(V + E) = O(R·C)
Extra memory	O(R·C): frontier can span the maze width	O(R·C) worst case: usually a smaller frontier
Finds a path if one exists	Yes	Yes
Typical use	Shortest route, nearest target	Reachability, maze generation, backtracking

10.2 Measured results

To check the theory, the engine was compiled with g++ -O2 and run on 1,000 freshly generated 21 × 21 mazes, calling both solvers on each. The figures below are averages.

Measurement	BFS	DFS
Average path length (steps)	45.1	70.5
Mazes with no solution found	0 / 1000	0 / 1000
Mazes where DFS path was shorter than BFS	—	0 / 1000

On average the DFS route was about 56% longer than the BFS route, and it was never shorter. Average open cells per maze were 213.6 of 441. These results match the theory: both solvers always succeed, but only BFS is optimal.

10.3 Which solver to use

BFS is the right choice whenever the shortest route matters, as in the game's hint feature. DFS is simpler in memory behaviour on very deep mazes and is the natural fit for generation and exploration, but its route should not be presented as optimal. The web interface reflects this: BFS results are labelled "Guaranteed Shortest Path" and DFS results "Exploration Path".

11. Supporting Structures: Server and Web Client

The data-structure focus of the project is the engine, but the surrounding code also relies on simple structures.

11.1 Server side

- **Strings and stringstream.** mazeToJSON and pathToJSON walk the grid or path and append text to a stringstream, producing output such as {"grid":[[1,1,...],...],"start":[1,1],"finish":[19,19]}.
- **Request parsing.** The first line of the HTTP request is split into method and path using a stringstream; routing is a chain of comparisons against /api/maze, /api/new, /api/solve and the static files.
- **Shared state.** One Maze object is passed by reference to startServer. The server handles one connection at a time in a blocking accept loop, so no locking is needed. The trade-off is that requests are served sequentially.
- **Buffers.** A 4096-byte char array receives each request, and sendResponse loops until every byte of the response string has been sent.

11.2 Browser side

- The grid arrives as a JavaScript array of arrays and is drawn cell by cell on a canvas.
- playerTrail is an array of visited coordinates used to draw the blue footprint line; solutionPath is an array filled one element at a time to animate the solver's path, with a 35 ms pause per step.
- Player movement checks array bounds and grid[r][c] === 1 before moving, mirroring Maze::isValid.

12. Console Interface

main_console.cpp provides a menu loop: display the maze, play manually with W/A/S/D, solve with BFS, solve with DFS, generate a new maze, or exit. The maze is shown as text, with # for walls, . for open cells, S for the start, E for the exit and P for the player. Input is validated: a failed cin read clears the error state and discards the line, so a stray letter at the menu does not cause an endless loop. Solver results are printed as the path length and the list of (row, col) coordinates.

13. Limitations and Future Scope

13.1 Limitations

- The server uses Winsock, so it builds only on Windows.
- The server is single-threaded and handles requests one after another.
- The solvers return only the final path; the order in which cells were explored is not recorded, so the web animation shows the route and not the search itself.
- The grid is a fixed 21 × 21; both `Maze` objects are created with these values in the entry-point files.
- `getContentType` is defined but unused, and static files are read from disk on every request.

13.2 Future scope

- Record the visit order in a `vector` and animate the explored cells, so the queue-versus-stack difference can be watched directly.
- Add Dijkstra's algorithm with a `priority_queue`, or A* with a Manhattan-distance heuristic, to extend the project to weighted and informed search.
- Replace the Winsock layer with a portable socket library or a thread pool to support other platforms and concurrent users.
- Allow the user to choose the maze size and the braiding ratio.

14. Conclusion

Maze Runner shows how the choice of data structure shapes an algorithm. One 2D vector serves as storage and graph. A recursive call stack builds the maze, a `std::stack` gives DFS its deep exploration, and a `std::queue` gives BFS its layered search and its shortest-path guarantee. A parent table of pairs turns either search into an actual route at the cost of one entry per cell. Measured over 1,000 mazes, BFS was always at least as short as DFS and 36% shorter on average (45.1 against 70.5 steps), matching the theory. The same engine drives both a console game and a web interface, which shows how a cleanly separated data-structure layer can be reused.

15. References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022 (graph search: BFS and DFS).
- E. Horowitz, S. Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C++, Universities Press.
- cppreference.com: std::vector, std::stack, std::queue, std::pair, std::shuffle, std::mt19937.
- Project source code: Maze.h, Maze.cpp, HttpServer.cpp, main_console.cpp, main_server.cpp, script.js.